003-School Inheritance

Difficulty: Easy Expect Time: 25 min **Homework**

Write a main program that creates one school of each type: **School**, **PrivateSchool**, and **PublicSchool**. Then, transfer students between them.

- **School**

  Member:

  Create a base class called **School** that has the following member variables and functions:

  - `std::string name`
    The name of the school (a string).
  - `int studentAmount`
    The current number of students in the school.
  - `int studentAmountNextYear`
    The number of students could have next year. This value should be set to the same as `studentAmount` when constructed.

  Member functions:

  - `void admissions(float amount)`
    Adds the given amount to `studentAmount`, but only if the `amount` is non-negative.
  - `void dropouts(float amount)`
    Subtracts the given amount from `studentAmount`, only if the `amount` is non-negative and less than or equal to the current `studentAmount.`
  - `void transfer(float amount, School& toSchool)`
    Transfers students by deducting the amount from the current school and adding it to the **toSchool**. This should be implemented by calling **dropouts(amount)** on the current school and `toSchool.admissions(amount)` on the destination school.

  Operator Overloading:

  - `operator<<`
    The `operator<<` should print the following format for each object:
    `name\tstudentAmount\tstudentAmountNextYear`
    where the fields are separated by a tab character (**\t**).

- **Private School**
  Also, create a class called **`PrivateSchool`** that is derived from **`School`.**
  In a **`PrivateSchool`**, one call to the **`dropouts()`** is allowed without penalty. After that, a penalty of 100 students is deducted from **`studentAmountNextYear`** each time the **`dropouts()`** is called. Therefore, the class must have a data member called **`dropoutCount`** to keep track of how many times **`dropouts()`** has been called, and it must override the **`dropouts()`.**

- **Public School**
  Finally, create a **`PublicSchool`** class derived from **`School`**.
  It should include a member variable called **`growing_rate`** (initialized to 0.05) and a member function **`apply_growth()`,** which increases **`studentAmountNextYear`** by updating:

$$studentAmountNextYear \mathrel{+}= growing_rate * studentAmountNextYear$$

  The result is rounded up using ceil before being added.

  When more than 100 students are removed in a single **`dropouts()`** call, the **`PublicSchool`** incurs a penalty: it loses 5% of **`studentAmountNextYear`**, rounded up using ceil. Override the **`dropouts()`** from the base class to implement this behavior.

- **Error Handling**:
  The following two error messages should be used for input validation:
  - **Invalid amount**
    Triggered when the input amount is less than or equal to 0, or is not an integer.
  - **Invalid student count**
    Triggered when an operation would cause **`studentAmount`** to become negative.

  In both cases, stop the operation and print the error message.

Input

1. Complete the required class implementation. The following files already contain some implementations
   a) **`School.h`**
   You can submit a header-only implementation, or split the implementation into **`*.cpp`** files.
2. The Online Judge system will replace the following files:
   a) **`entrypoint.cpp`**
   b) **`case.h`**
   c) **`test.h`**

3. The test cases for student available in **`case.h`**, but not all test data is included in that file. The Online Judge system will also include hidden test inputs.

4. Input range
   a) **amount** (argument to admissions, dropouts, transfer)
      i. $-1{,}000{,}000.0 <= amount <= 1{,}000{,}000.0$

Output

Online Judge will run the test:

1. **DON' T** output any information to the terminal.
2. When any test failed, Online Judge will try to list error messages.